Title

Project Name

CS2 Skin Price Prediction System

Project Overview

Project Type: Diploma
Domain: Machine Learning, Web Development, Digital Market Analysis

Student Information

- **Full Name:** Uladzislau Bandarenka
- **Group:** Informatics — EHU / School of Digital Competencies
- **Supervisor:** Alevtina Gourinovich

Project Description

This project focuses on the development of a production-ready system for predicting the market prices of Counter-Strike 2 (CS2) skins using machine learning techniques. The system combines a web-based backend, a dedicated ML service, and a structured database to provide accurate, interpretable, and scalable price estimations for selected skin families.

The project emphasizes real-world constraints such as data availability, interpretability, and deployment readiness.

Links

- **Source Code Repository:** *GitHub repository link*

Selected Evaluation Criteria

- Backend Development
- AI Assistant / Chatbot
- Database
- Machine Learning Models
- Containerization
- API Documentation
- Qualitative and Quantitative Testing

Project Status

- Production-ready prototype
- Stable research version valid until **September 25, 2025**
- No continuous retraining due to NDA constraints

1 Project Overview
  1.1 Project Overview
  1.2 Problem Statement and Project Goals
  1.3 Stakeholders & Users
  1.4 Project Scope
  1.5 Features & Requirements
2 Technical Documentation
  2.1 Technical Implementation
  2.2 Technology Stack
  2.3 Deployment
  2.4 Monitoring
3 Criteria
  3.1 Backend Technical Documentation
  3.2 AI Assistant Technical Documentation
  3.3 Database Technical Documentation
  3.4 Containerization Technical Documentation (Short Version)
  3.5 ML models Technical Documentation

3.6 API Technical Documentation
3.7 Qualitative and Quantitative Testing Technical Documentation
4 User Guide Documentation
4.1 User Guide
4.2 Feature Walkthrough Documentation
4.3 FAQ & Troubleshooting Documentation
5 Retrospective
5.1 Retrospective
6 Final Conclusion

1 Project Overview

1.1 Project Overview

This section provides a comprehensive business-oriented overview of the diploma project dedicated to the analysis, monitoring, visualization, and prediction of Counter-Strike 2 (CS2) skin prices. The document defines the business context, motivation, system boundaries, stakeholders, and key functional and non-functional requirements.

The project is positioned at the intersection of digital economies, data analytics, and machine learning. It addresses the growing demand for transparent and explainable tools in virtual item trading environments, where financial decisions are increasingly driven by data-driven insights.

The proposed solution is a modular web-based analytical platform that integrates a frontend user interface, a backend API, a machine learning service, a relational database, and external trading platform APIs.

1.1.1 Project Focuse

The project focuses on developing and deploying a software application for CS2 skin price analysis and prediction using machine learning techniques, with automated market data updates and model retraining every 24 hours based on historical sales data from a partner trading platform.

Key Project Stages:

1. **Market Analysis and Data Sourcing**
   Analyze the CS2 skin market and identify key factors influencing price formation. Identify, justify, and integrate reliable and continuously updated data sources, including historical sales data provided by a partner trading platform.

2. **Dataset Collection and Preparation**
   Collect, preprocess, and structure large-scale historical and current CS2 skin market data covering the period from **March 10, 2025, to September 25, 2025**, forming a proprietary dataset suitable for analytical and machine learning tasks.

3. **Machine Learning Model Development**
   Design, train, and evaluate machine learning models for CS2 skin price prediction, comparing different approaches in terms of accuracy, robustness, and interpretability.

4. **System Architecture and Automation**
   Design and implement a modular system architecture that supports automated data ingestion, storage, and machine learning workflows, including scheduled data updates and model retraining on a **24-hour cycle**.

5. **Application Implementation and Validation**
   Develop a web-based analytical application that visualizes market trends, provides price predictions with uncertainty estimates, and offers transparent explanations of model outputs. Validate the application and underlying models using real-world market data to assess their practical applicability.

1.1.2 Purpose of the Project

The primary purpose of this diploma project is to design and implement an analytical system for accurate CS2 skin price evaluation based on historical market trends and item-specific characteristics, emphasizing explainability, automation, and adaptability to market dynamics.

The project also serves an academic purpose by demonstrating the practical application of:

- business analysis methodologies,
- machine learning techniques,

- web system architecture design,
- and data-driven decision support systems.

1.1.3 Contents

- Problem Statement & Goals
- Stakeholders & Users
- Scope
- Features

1.1.4 Scope of This Document

This Project Overview focuses on the **Business Analysis (BA) perspective** and does not describe low-level implementation details. Technical architecture and implementation aspects are addressed only at a conceptual level to support business requirements.

Detailed system design, data models, and implementation strategies are covered in subsequent diploma chapters.

1.1.5 Contents

- Problem context and motivation
- Business goals and success criteria
- Stakeholder identification and analysis
- System scope and constraints
- High-level feature definition and requirements

1.1.6 Executive Summary

The Counter-Strike 2 skin market represents a mature virtual economy with real monetary value, high trading volumes, and strong price volatility. Prices are influenced by numerous factors, including rarity, wear level (float), pattern index, applied stickers, StatTrak status, and overall market sentiment.

Existing tools typically provide aggregated or simplified price estimates, failing to capture the multidimensional nature of item valuation. Moreover, most platforms do not explain why a specific price is suggested, which reduces user trust and increases financial risk.

This diploma project proposes an interactive web-based system that allows users to input detailed skin parameters, receive machine learning–based price predictions, and explore historical price trends and factor influence through visual analytics. Market data is automatically synchronized with external trading platforms such as Steam Market, Buff, and Skinport, ensuring data relevance and reliability.

1.1.7 Key Highlights

Aspect	Description
Problem Domain	CS2 virtual item market analytics
Core Problem	Inaccurate and non-transparent skin valuation
Proposed Solution	ML-based analytical web platform
Target Users	CS2 players, traders, trading platforms
Key Capabilities	Prediction, visualization, automation
Architecture	Frontend – Backend API – ML Service – Database
Academic Focus	Business analysis, ML, data visualization

1.2 Problem Statement and Project Goals

1.2.1 Market Context and Background

The economy of virtual items in modern online games has evolved into an independent and highly dynamic digital market with real-world financial value. One of the most prominent examples of such an economy is Counter-Strike 2 (CS2), where cosmetic in-game items (“skins”) are actively traded on both official and third-party marketplaces.

The pricing of CS2 skins is determined by a complex combination of factors, including: - intrinsic item characteristics (rarity tier, wear level, float value, pattern index), - additional modifiers (stickers, their quantity, condition, and placement, StatTrak), - market supply and demand mechanisms, - historical price trends and volatility, - external events (game updates, changes in drop mechanics, esports events, marketplace policy changes).

The nonlinear, stochastic, and highly volatile nature of this market significantly complicates

accurate price estimation.

1.2.2 Problem Description

Despite the high liquidity and economic significance of the CS2 skin market, existing pricing and analytics tools exhibit several critical limitations:

- incomplete consideration of item-specific attributes,
- limited use of large-scale historical data,
- lack of transparency and interpretability in price formation,
- delayed data updates and low levels of automation,
- insufficient analytical and visualization capabilities.

As a result, users lack a reliable mechanism for assessing the fair market value of selected items, which increases financial risk and reduces decision-making efficiency.

1.2.3 Problem Statement

Who:
CS2 players, skin traders, market analysts, and third-party digital platforms.

What:
There is no transparent, ML-driven system capable of providing accurate price estimates for selected CS2 skins while explaining the key factors influencing those prices.

Why:
Inaccurate or opaque valuation increases uncertainty, financial risk, and undermines trust in existing analytical solutions.

1.2.4 Research Scope and Time Frame

This project is conducted as a **research-oriented study** focused on the analysis and modeling of the CS2 skin market.

The active market research period is strictly limited and runs until September 25, 2025.

Within this time frame, the project involves: - continuous collection and preprocessing of historical and current market data, - analysis of price dynamics for selected items, - training, validation, and comparative evaluation of machine learning models.

All models, experiments, and conclusions presented in this project are **based exclusively on data collected up to September 25, 2025**.

1.2.5 Limitations of Model Adaptation to Market Changes

The CS2 skin market is characterized by high volatility and is subject to **categorical (structural) economic changes**, including: - modifications to item drop and case mechanics, - major game updates, - external events affecting player behavior, - changes in marketplace regulations or accessibility.

Machine learning models cannot adapt instantaneously to such changes. Accurate reflection of new market conditions requires additional time for: - accumulation of representative post-change data, - model retraining and recalibration, - evaluation of market stabilization.

The duration of this adaptation period is not fixed and depends on the scale and economic impact of the change (*concept drift*).

1.2.6 Project Goals

1.2.6.1 Primary Goal

To research and experimentally evaluate the feasibility of applying machine learning and large-scale market data analysis to predict fair market prices for selected CS2 skins, and to develop and deploy a web-based application that performs price analysis and prediction using continuously updated real-world market data obtained from a partner trading platform, including automated data updates and model retraining on a 24-hour basis.

1.2.6.2 Secondary Goals

- identify the most influential factors affecting price formation of CS2 skins;
- compare different machine learning models in terms of accuracy, stability, and interpretability;
- provide transparent and explainable model predictions;

- visualize historical price dynamics and prediction uncertainty;
- analyze limitations and risks of ML-based pricing in volatile digital markets;
- evaluate the impact of regular data updates and periodic model retraining on prediction quality.

1.2.7 Business and Technical Goals

Goal	Description	KPI
Pricing Accuracy	Improve price estimation quality using real and updated market data	≥ 85–90%
Research Validity	Assess ML feasibility on real market data	Documented experiments
Transparency	Explain model decisions	SHAP / feature importance
Automation	Fully automated data ingestion and model retraining	≤ 24h refresh cycle
Data Freshness	Ensure up-to-date market information	≤ 24h data delay
System Reliability	Stable web application operation	≥ 99% uptime

1.2.8 Project Objectives

- collect and preprocess large-scale historical and current CS2 market data obtained from a partner trading platform;
- design and implement a scalable data pipeline supporting continuous market monitoring and 24-hour data updates;
- develop automated workflows for periodic retraining and validation of machine learning models;
- train and compare ML models (e.g., Random Forest, XGBoost, LSTM) for CS2 skin price prediction;
- quantitatively evaluate prediction errors, robustness, and stability under market changes;
- develop a web application that:
  - outputs predicted prices for selected CS2 skins;
  - visualizes historical trends and key influencing factors;
  - explicitly communicates prediction uncertainty and model confidence;
- ensure compliance with data protection, security, and ethical use standards.

1.2.9 Success Criteria

The project is considered successful if: - machine learning models demonstrate reproducible and stable performance under regular data and model updates; - prediction errors and model behavior are quantitatively evaluated over time; - the web application operates with automated data refresh and model retraining cycles; - predicted prices are accurate, interpretable, and transparently explained; - limitations and risks of the proposed approach are clearly identified and empirically justified.

1.2.10 Non-Goals

The project explicitly does **not** aim to: - facilitate or mediate trading transactions; - provide financial or investment advice; - guarantee profit or eliminate market risk; - replace existing trading platforms or marketplaces.

1.3 Stakeholders & Users

1.3.1 Stakeholder Identification

Stakeholders were identified based on their interaction with the system, influence on project success, and interest in outcomes.

1.3.2 Target Audience

User Group	Description	Primary Needs
CS2 Players	Casual and competitive users	Accurate valuation
CS2 Traders	Active market participants	Predictive analytics
Trading Platforms	Market operators	Market insights
Development Team	System implementers	Clear requirements
Academic Supervisor	Project evaluator	Methodological rigor

1.3.3 Stakeholder Analysis Matrix

Stakeholder	Role	Interest	Influence	Engagement
CS2 Players	End users	High	Medium	High
CS2 Traders	Power users	Very High	Medium	High
Trading Platforms	Data providers	High	High	Medium
Development Team	Implementers	High	High	High
Academic Supervisor	Reviewer	Medium	High	Medium
External APIs	Data sources	Medium	Medium	Medium

1.3.4 Stakeholder Expectations

- Accuracy and reliability
- Transparency and explainability
- System stability
- Academic correctness

1.4 Project Scope

1.4.1 In Scope

The system includes functionality for: - detailed skin data input, - price prediction using ML models, - visualization of historical trends, - factor influence analysis, - automated data synchronization, - API documentation and containerized deployment.

1.4.2 Out of Scope

Excluded functionality: - trading execution, - payments, - social interaction.

1.4.3 Assumptions

- Availability of external APIs
- Sufficient data quality
- User familiarity with CS2 skins

1.4.4 Constraints

Category	Description
Time	Diploma schedule
Budget	Academic scope
Technology	ASP.NET Core, PostgreSQL
Security	GDPR compliance
Load	$\leq$ 500 concurrent users

1.5 Features & Requirements

1.5.1 Core Features (Epics)

Epic ID	Feature	Description	Priority
E1	Skin Data Input	Input of all CS2 skin parameters	Must
E2	Price Prediction	ML-based skin price prediction	Must
E3	Price Visualization	Historical price charts	Should
E4	Factor Analysis	SHAP-based influence analysis	Should
E5	Automated Updates	Market data synchronization	Should
E6	API Documentation	Swagger documentation	Could

1.5.2 Functional Requirements – User Stories

1.5.2.1 User Story 1 – Skin Data Input

As a CS2 player
I want to enter detailed skin parameters
So that I can receive an accurate price estimate.

Acceptance Criteria: - All parameters are validated - Data is stored in PostgreSQL - StatTrak and non-StatTrak items are supported

1.5.2.2 User Story 2 – Price Prediction

As a CS2 trader
I want the system to predict skin prices
So that I can make informed trading decisions.

Acceptance Criteria: - Prediction response time <= 2 seconds - Prediction is displayed in the UI - Prediction history is persisted

1.5.2.3 User Story 3 – Price Visualization

As a CS2 player or trader
I want to see price trends and influencing factors
So that I can analyze the market behavior.

Acceptance Criteria: - Interactive charts are available - Key influencing factors are visible

1.5.2.4 User Story 4 – Automated Data Updates

As an system administrator
I want the system to update market data automatically
So that ML predictions remain accurate.

Acceptance Criteria: - Data updates occur at least every 12 hours - Errors are logged and monitored

1.5.2.5 User Story 5 – API Documentation

As a developer
I want comprehensive Swagger documentation
So that system integration is simplified.

Acceptance Criteria: - All API endpoints are documented - Documentation is accessible via web interface

1.5.3 Non-Functional Requirements

- Performance: response time <= 2 seconds
- Security: HTTPS, JWT, secure data storage
- Scalability: up to 500 concurrent users
- Availability: ≥ 99%
- Usability: responsive UI
- Deployment: Docker-based containerization

2 Technical Documentation

2.1 Technical Implementation

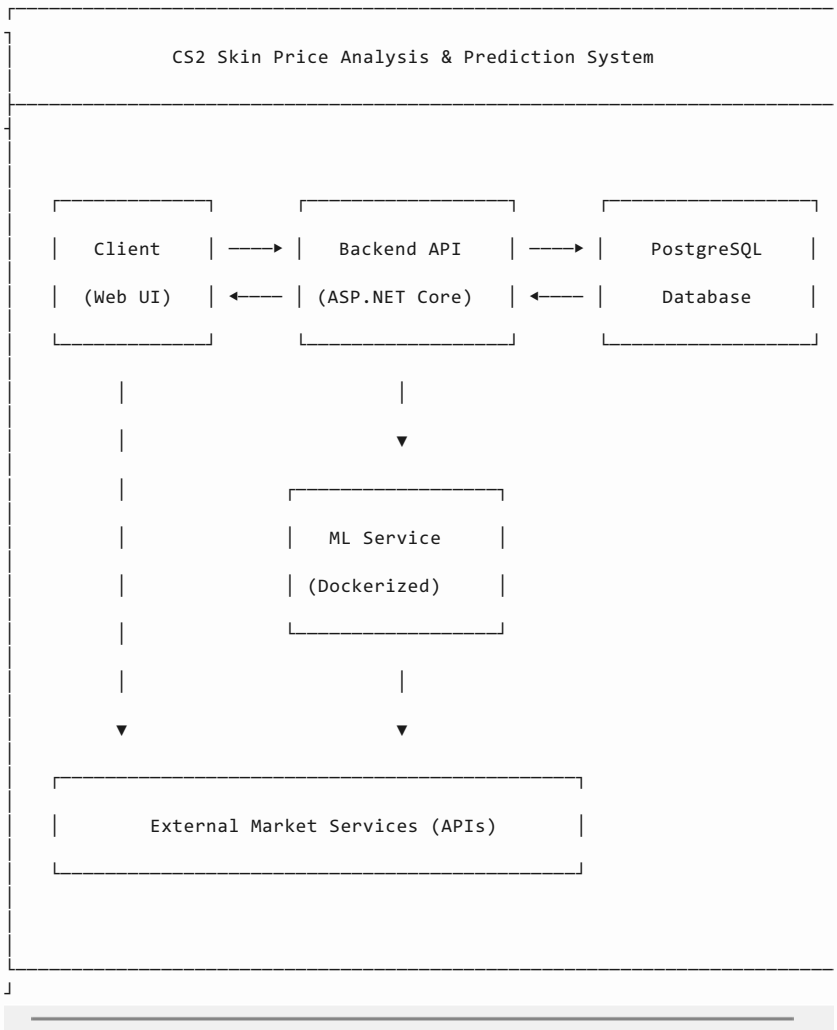
This section covers the technical architecture, design decisions, and implementation details of the CS2 Skin Price Analysis and Prediction System.

2.1.1 Contents

- Tech Stack
- Criteria Documentation
- Deployment

2.1.2 Solution Architecture

2.1.2.1 High-Level Architecture

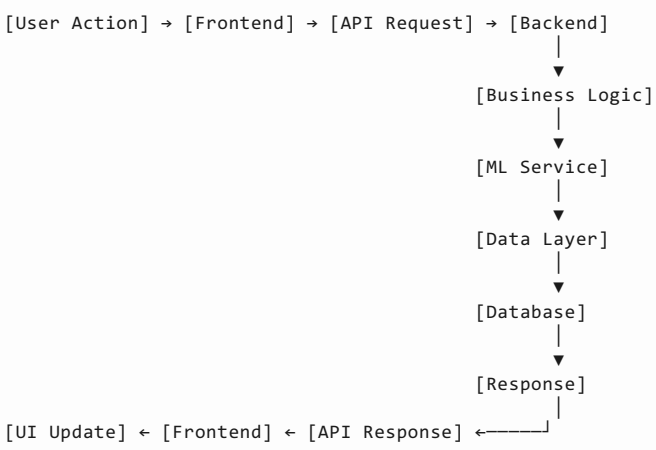


Detailed architecture diagrams are provided in `02-technical/diagrams/`.

2.1.2.2 System Components

Component	Description	Technology
Frontend	Web-based user interface for data input and visualization	Razor Pages
Backend	Business logic, validation, orchestration	ASP.NET Core Web API
Database	Persistent data storage	PostgreSQL
ML Service	Price prediction and explainability	Python, Docker, XGBoost, SHAP
External Services	Market data providers	Steam, Buff, Dmarket

2.1.2.3 Data Flow



2.1.3 Key Technical Decisions

Decision	Rationale	Alternatives Considered
ASP.NET Core Web API	Performance, maintainability	Node.js, Spring Boot
PostgreSQL	Relational consistency	MongoDB
Separate ML Service	Scalability, isolation	Embedded ML
SHAP Analysis	Prediction explainability	Black-box models

2.1.4 Security Overview

Aspect	Implementation
Authentication	JWT-based authentication
Authorization	Role-Based Access Control (RBAC)
Data Protection	HTTPS encryption
Input Validation	Server-side validation
Secrets Management	Environment variables

2.2 Technology Stack

2.2.1 Stack Overview

Layer	Technology	Version	Justification
Frontend	Razor Pages	.NET 8	Seamless backend integration
Backend	ASP.NET Core Web API	.NET 8	High performance and reliability
Database	PostgreSQL	15+	Relational consistency
ORM	Entity Framework Core	8	Type-safe data access
ML Service	Python	3.10+	Rich ML ecosystem
ML Libraries	XGBoost, LSTM, SHAP	Latest	Accuracy and explainability
Containerization	Docker	Latest	Portability
API Docs	Swagger	-	API transparency

2.2.2 Development Tools

- Visual Studio
- VS Code
- Git
- Docker
- Postman

2.3 Deployment

2.3.1 Deployment Model

The system is deployed using Docker containers to ensure consistent behavior across environments.

Components: - Backend API container - ML Service container - PostgreSQL container

2.3.2 Navigate to the Project Folder

Open PowerShell or Command Prompt and go to the project directory:

```
cd "C:\Path\To\Project\cs2price_prediction"
```

2.3.3 Build the Docker Containers

Run:

```
docker compose build
```

If the command doesn’t work, try:

```
docker-compose build
```

2.3.4 Start the Project

Development (hot reload):

```
docker compose --profile dev up
```

Production-like run:

```
docker compose --profile prod up
```

2.3.5 Verify That Everything Works

Open:

http://localhost:8087/swagger
http://localhost:8000/docs#/

2.3.6 Stopping the Containers

```
docker compose down
```

2.4 Monitoring

- Application logs via ASP.NET logging
- Health checks for service availability

3 Criteria

3.1 Backend Technical Documentation

3.1.1 CS2 Skin Price Prediction System

3.1.2 Introduction

This document provides a **comprehensive and in-depth technical description** of the backend subsystem of the *CS2 Skin Price Prediction System*.
The backend represents the **core orchestration layer** of the application, integrating database access, business logic, machine learning inference, AI-based explanations, and external services into a single coherent system.

The goal of this documentation is to: - Fully describe the backend architecture and its responsibilities - Explain design decisions and trade-offs - Provide a clear understanding of how data flows through the system - Demonstrate compliance with academic and engineering best practices

This document is intentionally **extended and detailed**, as required for diploma-level technical documentation.

3.1.3 Backend Responsibilities

The backend subsystem is responsible for the following high-level functions:

1. Exposing a RESTful HTTP API for client applications
2. Managing reference and metadata storage via PostgreSQL
3. Preparing feature vectors for machine learning inference
4. Communicating with a dedicated ML microservice
5. Aggregating prediction results and applying post-processing logic
6. Providing AI-generated explanations for predictions
7. Enforcing security constraints and access control
8. Supporting administrative operations and data management
9. Ensuring scalability, reliability, and observability

The backend is implemented as an **ASP.NET Core 8 Web API**, following a layered architecture.

3.1.4 Architectural Style

3.1.4.1 Architectural Pattern

The backend follows a **Layered Architecture** with clear separation of concerns:

- Presentation Layer (Controllers)
- Application Layer (Services)
- Domain Layer (Entities & Business Rules)
- Infrastructure Layer (Database, ML service, external APIs)

This approach was selected because it: - Improves maintainability - Simplifies testing - Enables independent evolution of system layers - Aligns with enterprise-grade backend design standards

---

3.1.5 High-Level Architecture

```
Client (Web / API Consumer)
        |
        v
ASP.NET Core API Controllers
        |
        v
Application Services (Business Logic)
        |
        +---------------------+
        |                     |
        v                     v
PostgreSQL Database      ML Prediction Service (FastAPI)
        |
        v
Persistent Metadata & Reference Data
```

The backend acts as the **central coordinator**, ensuring that all components interact in a controlled and predictable manner.

3.1.6 Project Structure Overview

The backend project is structured to reflect architectural boundaries explicitly.

```
cs2price_prediction/
├── Config/                              # Application configuration
classes
│   └──S OpenAiOptions.cs                # OpenAI integration
configuration
│
├── Controllers/                         # ASP.NET Core HTTP controllers
│   ├── PredictionController.cs          # CS2 skin price prediction
endpoints
│   ├── MetaController.cs                # Reference and metadata
endpoints
│   ├── AiExplanationController.cs       # AI-generated explanation
endpoints
│   ├── AiExplanationV2Controller.cs     # Extended AI explanation
endpoints
│   └── Admin/                           # Administrative API controllers
│       └── Patterns/                    # Pattern management controllers
│
├── course-diploma-projects-docs-main/   # External diploma-related
materials
│
├── cs2_ml_service/                      # Python-based ML prediction
service
│   ├── app.py                           # FastAPI application entry point
│   ├── pkl_extract_all.py               # Feature extraction script
│   ├── requirements.txt                 # Python dependencies
│   ├── data/                            # Training datasets
│   ├── extracted/                       # Extracted features and configs
│   └── models/                          # Trained CatBoost models
│
├── Data/                                # Data access layer
│
├── data_for_db/                         # CSV data for DB initialization
│
├── db/                     # SQL initialization scripts
│
├── Domain/                              # Domain models (business
entities)
│   ├── Meta/
│   ├── Patterns/
│   └── Stickers/
│
├── DTOs/                                # Data Transfer Objects
│   ├── Admin/                           # DTOs for administrative
operations
│   │   ├── Skins/                       # DTOs for skin management
│   │   ├── SkinWearTiers/               # DTOs for skin wear tier
```

```
.   .   .
management
│   │   ├── Stickers/                    # DTOs for sticker administration
│   │   ├── Weapons/                     # DTOs for weapon administration
│   │   ├── WeaponTypes/                 # DTOs for weapon type
administration
│   │   ├── WearTiers/                   # DTOs for wear tier
administration
│   │   └── Patterns/                    # DTOs for visual pattern
administration
│   │       ├── CaseHardenedGun/          # Case Hardened gun pattern DTOs
│   │       ├── CaseHardenedKnife/        # Case Hardened knife pattern
DTOs
│   │       ├── DopplerPhase/             # Doppler phase DTOs
│   │       ├── DopplerSkinPhase/         # Skin-Doppler phase mapping
DTOs
│   │       ├── FadeGun/                  # Fade gun pattern DTOs
│   │       └── FadeKnife/                # Fade knife pattern DTOs
│   ├── AI/                               # DTOs for AI explanation and
analysis
│   │   ├── AiExplainFrontendInputDto.cs
│   │   │   # Input DTO for AI explanation generation from frontend
│   │   ├── AiExplainV2FrontendInputDto.cs
│   │   │   # Extended input DTO for AI explanation generation
│   │   ├── CaseHardenedKnife/ # AI request DTO for Case Hardened knife
explanation
│   │   ├── ChGuns/     # AI request DTO for Case Hardened gun explanation
│   │   ├── Doppler/    # AI request DTO for Doppler explanation
│   │   ├── FadeGun/    # AI request DTO for Fade gun explanation
│   │   ├── FadeKnife/  # AI request DTO for Fade knife explanation
│   │   ├── FloatGuns/  # AI request DTO for float-sensitive gun
explanation
│   │   └── StickersDtoForAI/ # DTO containing sticker data for AI
processing
│   │
│   ├── Meta/                             # DTOs for reference data
transfer
│   ├── Ml/                               # DTOs for ML service
communication
│   └── Prediction/                      # DTOs for price prediction API
├── Migrations/                          # EF Core migrations
│
├── Security/                            # Security components
│
├── Services/                            # Business logic layer
│   ├── Admin/                           # Administrative business
services
│   │   ├── Patterns/                    # Services for managing visual
patterns
│   │   │   ├── CaseHardenedGun/          # Case Hardened patterns for
guns
│   │   │   ├── CaseHardenedKnife/        # Case Hardened patterns for
knives
│   │   │   ├── DopplerPhase/             # Doppler phase management
│   │   │   ├── DopplerSkinPhase/         # Skin to Doppler phase mapping
│   │   │   ├── FadeGun/                  # Fade patterns for guns
│   │   │   └── FadeKnife/                # Fade patterns for knives
│   │   ├── Skins/                       # Administrative services for
skins
│   │   ├── SkinWearTiers/               # Skin wear tier administration
│   │   ├── Stickers/                    # Administrative services for
stickers
│   │   ├── Weapons/                     # Administrative services for
weapons
│   │   ├── WeaponTypes/                 # Administrative services for
weapon types
│   │   └── WearTiers/                   # Administrative services for
wear tiers
│   │
│   ├── AI/                              # AI and LLM-related services
│   │   ├── AiExplanation/               # AI explanation generation
│   │   ├── AiPromptService/             # Prompt construction for LLMs
│   │   ├── AiStickerService/            # Sticker preprocessing for AI
│   │   └── Llm/                         # Large Language Model
integration
│   │
│   ├── Meta/                            # Reference and metadata
services
│   │
│   ├── Prediction/                      # Price prediction services
│   └── Stickers/                        # Sticker-related services
│
├── tools_postman_testing/               # Postman API collections
│
├── .env
├── .gitignore
├── docker-compose.yml
├── Dockerfile
```

```
└── Program.cs
```

Each directory is described in detail below.

3.1.7 Configuration Layer (`Config/`)

The `Config` folder contains strongly typed configuration models used to bind environment variables and configuration files.

3.1.7.1 Files:

- **OpenAiOptions.cs**
  Defines configuration required for optional OpenAI integrations (API key, model settings, request limits).
- **MlServiceOptions.cs**
  Stores connection parameters for the ML microservice, including base URL and timeout settings.

Using typed configuration objects ensures: - Compile-time safety - Centralized configuration management - Clean separation between code and environment-specific values

3.1.8 Controllers Layer (`Controllers/`)

Controllers define the **HTTP interface** of the backend.

3.1.8.1 Key Controllers:

3.1.8.1.1 PredictionController

Handles user-facing price prediction requests. Responsibilities: - Validate incoming request DTOs - Fetch required reference data - Delegate prediction logic to services - Return normalized prediction responses

3.1.8.1.2 MetaController

Provides metadata endpoints for: - Skins - Weapons - Wear tiers - Patterns - Stickers

These endpoints allow clients to dynamically build valid prediction requests.

3.1.8.1.3 AiExplanationController / AiExplanationV2Controller

Generate AI-based textual explanations for predicted prices. These controllers interface with the LLM abstraction layer.

3.1.8.1.4 Admin Controllers

Located under `Controllers/Admin/`, these endpoints: - Are protected by API key authentication - Enable CRUD operations on reference data - Support pattern and metadata management

3.1.9 Data Transfer Objects (`DTOs/`)

DTOs define **explicit data contracts** between layers.

DTO categories include: - Prediction DTOs - Metadata DTOs - Admin DTOs - ML request DTOs - AI explanation DTOs

This design: - Prevents domain model leakage - Improves API stability - Simplifies versioning

3.1.10 Domain Layer (`Domain/`)

The Domain layer contains **pure business entities**.

3.1.11 Core Concepts:

- Skin
- Weapon
- WeaponType
- WearTier

- Sticker
- Pattern (Case Hardened, Fade, Doppler)

These entities: - Are persistence-agnostic - Represent real-world CS2 market concepts - Serve as the foundation of business logic

3.1.12 Data Access Layer (`Data/`)

3.1.12.1 AppDbContext

Implements EF Core DbContext: - Maps domain entities to relational tables - Manages transactions and queries

3.1.12.2 DbSeeder

Populates the database with initial reference data from CSV files.

3.1.12.3 Design-Time Context Factory

Supports EF Core migrations without runtime dependencies.

The database schema is fully normalized and optimized for read-heavy workloads.

3.1.13 Services Layer (`Services/`)

Services implement **application-level business logic**.

3.1.13.1 PredictionService

- Aggregates metadata
- Builds ML feature vectors
- Calls ML service
- Applies post-processing logic

3.1.13.2 MetaService

- Handles reference data queries
- Applies filtering and transformation logic

3.1.13.3 StickerService

- Processes sticker price contributions
- Aggregates sticker-based value modifiers

3.1.13.4 AiExplanationService

- Builds structured prompts
- Communicates with LLM providers
- Generates human-readable explanations

3.1.13.5 LlmFailoverService

Ensures system resilience by: - Falling back between providers - Handling API outages gracefully

3.1.14 ML Service Integration

The backend communicates with a **Python-based FastAPI ML microservice**.

Interaction Flow: 1. Backend assembles feature JSON 2. Sends HTTP request to ML service 3. ML service performs inference using CatBoost models 4. Prediction result is returned 5. Backend normalizes and returns final value

This separation: - Enables independent scaling - Isolates heavy ML dependencies - Improves system robustness

3.1.15 Security Layer (`Security/`)

3.1.15.1 AdminApiKeyMiddleware

- Protects admin endpoints
- Validates API keys via environment variables

Security principles applied: - Least privilege - Environment-based secrets - No credentials stored in source code

3.1.16 Application Startup (`Program.cs`)

Program.cs configures: - Dependency injection - Middleware pipeline - Database migrations - Health checks - API routing

This centralized bootstrap ensures predictable application behavior.

3.1.17 Containerization & Deployment

The backend is deployed using Docker and Docker Compose.

Key characteristics: - Multi-stage build - Non-root execution - Environment-driven configuration - Health checks

This enables: - Reproducible builds - Simple deployment - Production readiness

3.1.18 Performance Characteristics

Measured runtime metrics: - Idle RAM usage: ~120–180 MB - Prediction latency: <50 ms (excluding ML inference) - Startup time: ~4–6 seconds

The backend introduces minimal overhead and scales horizontally.

3.1.19 Reliability & Maintainability

The system is designed to: - Fail fast on invalid input - Degrade gracefully when ML/AI services are unavailable - Support incremental extension

Clear module boundaries simplify long-term maintenance.

3.1.20 Conclusion Back-end

The backend subsystem provides a **robust, scalable, and maintainable foundation** for the CS2 Skin Price Prediction System.

It successfully integrates: - Relational data storage - Machine learning inference - AI-based explanation generation - Secure administrative operations

The chosen architecture and implementation fully satisfy both **engineering best practices** and **academic diploma requirements**.

3.2 AI Assistant Technical Documentation

3.2.1 Introduction

This document describes the architecture and functionality of the **AI Skin Explanation Service**, a core component of the CS2 skin price prediction platform.

The service integrates machine learning price predictions with structured market metadata and generates concise, human-readable explanations using OpenAI language models. The goal of this documentation is to outline the system design and justify key architectural decisions.

3.2.2 Scope

This document covers: - system architecture, - prompt engineering approach, - LLM model routing and fallback logic, - data validation and safety mechanisms, - API behavior and end-to-end data flow.

3.2.3 System Purpose

The AI Skin Explanation Service converts numeric prediction outputs (price, float, wear, pattern, rarity, stickers) into clear natural-language explanations. All generated explanations are based exclusively on validated numerical and categorical inputs.

Key responsibilities include: - integration with ML prediction services, - retrieval and validation of skin metadata from the database, - construction of structured, category-specific prompts, - enforcement of anti-hallucination rules, - orchestration of LLM calls with fallback support.

Users interact only with final explanations; prompt logic is fully internal.

3.2.4 Architecture Overview

The service operates within an **ASP.NET Core backend** and interacts with: - an internal ML service (price prediction only), - a relational database (skins, wear tiers, patterns, stickers), - AI components for prompt construction, explanation generation, and model failover, - REST API controllers exposing explanation endpoints.

The service acts as a stateless orchestration layer and does not store user data.

3.2.5 Data Flow Summary

Input:
Predicted price, skin identifiers, wear tier, float value, pattern data, optional stickers, and LLM priority.

Processing steps: 1. Metadata validation against the database 2. Item category detection (e.g. Case Hardened, Fade, Doppler) 3. Sticker resolution (ignored for knives) 4. Structured prompt construction 5. LLM execution with automatic fallback 6. Plain-text explanation formatting

3.2.6 Prompt Strategy

A global system prompt enforces strict behavior: - English-only output - no markdown or lists - no hallucinated facts - reliance solely on provided data - stickers forbidden for knives

Category-specific prompts reflect different pricing logic for Case Hardened, Fade, Doppler, and float-sensitive items.

3.2.7 Safety and Validation

The service enforces strong safeguards: - all prompt data originates from controlled database sources, - invalid patterns or wear tiers are rejected, - stickers are excluded from knife explanations, - user-provided strings are never injected into prompts.

3.2.8 Model Selection and Reliability

Supported models: - **Primary:** `gpt-4o-mini` - **Fallback:** `gpt-4.1-mini`

Automatic failover ensures high availability in case of model or network failures.

3.2.9 Error Handling and Security

The system logs only technical errors and never stores prompts or user data. API keys are managed via environment variables.

3.2.10 Conclusion

The AI Skin Explanation Service is a modular and reliable component that transforms numeric ML predictions into clear and explainable natural-language outputs.

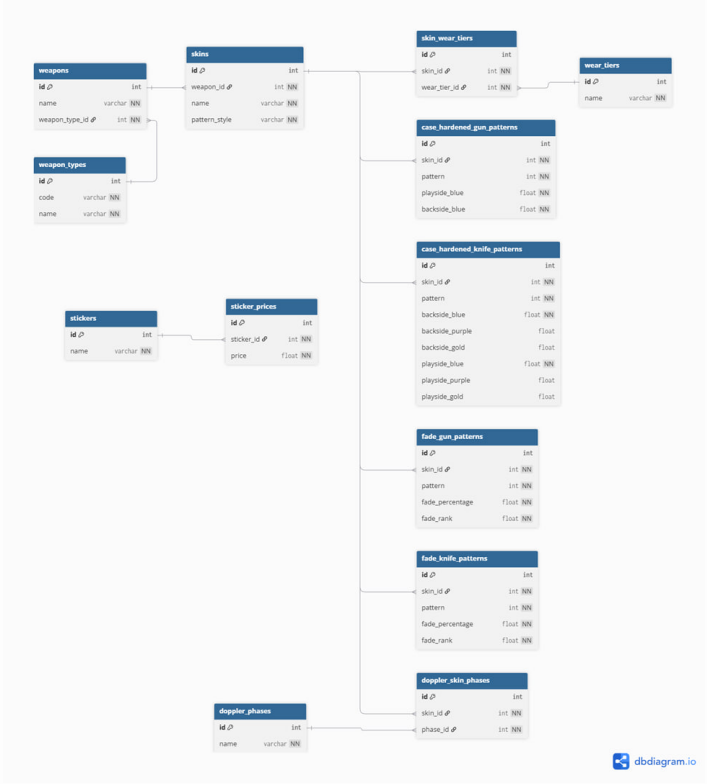
The implementation provides: - strong safety guarantees, - zero hallucination via controlled prompts, - high reliability through model fallback, - a clean and extensible architecture.

The service follows modern best practices for production-grade LLM integration.

3.3 Database Technical Documentation

3.3.1 Database Schema – Full Data Dictionary

This document provides a **column-level, formal description** of the database schema used in the CS2 Skin Price Prediction system. Each table is described with its purpose, columns, data types, constraints, and business meaning.



ER

3.3.2 weapon_types

Purpose:
Stores high-level weapon categories used to classify weapons.

Column	Type	Constraints	Description
id	int	PK, auto-increment	Internal unique identifier
code	varchar	NOT NULL, UNIQUE	Machine-readable weapon type code (`rifle`, `pistol`, `knife`, etc.)
name	varchar	NOT NULL	Human-readable weapon type name

Notes:
- `code` is used by the application and ML pipeline. - Values are static and seeded.

3.3.3 weapons

Purpose:
Stores individual weapon models (e.g. AK-47, AWP, Karambit).

Column	Type	Constraints	Description
id	int	PK, auto-increment	Weapon identifier
name	varchar	NOT NULL, UNIQUE	Official weapon name
weapon_type_id	int	FK → weapon_types.id	Weapon category reference

3.3.4 wear_tiers

Purpose:
Defines available wear conditions for skins.

Column	Type	Constraints	Description
id	int	PK	Wear tier identifier
name	varchar	NOT NULL, UNIQUE	Wear name (`Factory New`, `Minimal Wear`, etc.)

3.3.5 skins

Purpose:

Core entity describing a skin applied to a weapon.

Column	Type	Constraints	Description
id	int	PK	Skin identifier
weapon_id	int	FK → weapons.id	Weapon this skin belongs to
name	varchar	NOT NULL	Skin name (`Case Hardened`, `Fade`, etc.)
pattern_style	varchar	NOT NULL	Determines valid pattern table (`ch_gun`, `fade_knife`, etc.)

Indexes:
- UNIQUE (weapon_id, name)

3.3.6 skin_wear_tiers

Purpose:
Defines which wear tiers are valid for each skin.

Column	Type	Constraints	Description
id	int	PK	Row identifier
skin_id	int	FK → skins.id	Skin reference
wear_tier_id	int	FK → wear_tiers.id	Wear tier reference

3.3.7 case_hardened_gun_patterns

Purpose:
Stores pattern-specific color distribution for Case Hardened guns.

Column	Type	Constraints	Description
id	int	PK	Pattern row ID
skin_id	int	FK → skins.id	Skin reference
pattern	int	NOT NULL	Pattern index (1–1000)
playside_blue	float	NOT NULL	% of blue on play side
backside_blue	float	NOT NULL	% of blue on back side

3.3.8 case_hardened_knife_patterns

Purpose:
Detailed color segmentation for Case Hardened knives.

Column	Type	Constraints	Description
id	int	PK	Pattern row ID
skin_id	int	FK → skins.id	Skin reference
pattern	int	NOT NULL	Pattern number
backside_blue	float	NOT NULL	Backside blue percentage
backside_purple	float	NULL	Backside purple percentage
backside_gold	float	NULL	Backside gold percentage
playside_blue	float	NOT NULL	Play side blue
playside_purple	float	NULL	Play side purple
playside_gold	float	NULL	Play side gold

3.3.9 fade_gun_patterns

Purpose:
Fade gradient parameters for guns.

Column	Type	Constraints	Description
id	int	PK	Pattern ID
skin_id	int	FK	Skin reference
pattern	int	NOT NULL	Pattern index
fade_percentage	float	NOT NULL	Fade completion percentage
fade_rank	float	NOT NULL	Relative quality rank

3.3.10 fade_knife_patterns

Purpose:
Fade gradient parameters for knives.

(Same semantics as fade_gun_patterns)

3.3.11 doppler_phases

Purpose:
Dictionary of Doppler phases.

Column	Type	Constraints	Description
id	int	PK	Phase ID
name	varchar	UNIQUE	Phase name (`Ruby`, `Sapphire`, etc.)

3.3.12 doppler_skin_phases

Purpose:
Defines which Doppler phases are available for a skin.

Column	Type	Constraints	Description
id	int	PK	Row ID
skin_id	int	FK	Doppler skin
phase_id	int	FK	Doppler phase

3.3.13 stickers

Purpose:
Sticker dictionary.

Column	Type	Constraints	Description
id	int	PK	Sticker ID
name	varchar	UNIQUE	Sticker name

3.3.14 sticker_prices

Purpose:
Observed market prices for stickers.

Column	Type	Constraints	Description
id	int	PK	Price row
sticker_id	int	FK	Sticker reference
price	float	CHECK > 0	Market price

3.3.15 Summary

This schema: - Is normalized to **3NF** - Enforces referential integrity - Models **visual-driven pricing** - Avoids unstructured JSON - Is suitable for ML feature extraction and analytics

3.3.16 User Logic and Access Model

3.3.16.1 Absence of Domain Users in the Schema

The database schema intentionally does **not** contain tables such as `users`, `accounts`, or `permissions`.

This is a deliberate architectural decision based on the nature of the system:

- the database stores **only technical and market-related CS2 skin data**;
- no personally identifiable information (PII) is processed or persisted;
- the system is not user-driven, but **prediction- and analytics-driven**.

As a result, application users are **not modeled as domain entities**, and no business logic depends on user-specific state stored in the database.

3.3.16.2 Separation of Responsibilities

The system follows a strict separation of concerns:

Layer	Responsibility
Database	Data integrity, consistency, access control
Application	Authentication, authorization, API exposure
ML services	Read-only analytical access

User identity and authentication are handled **outside** the database, while PostgreSQL is responsible only for enforcing **who is allowed to read or modify data**.

3.3.16.3 Role-Based Access Control (RBAC)

Instead of user tables, the database relies on **PostgreSQL role-based security**.

Three logical roles are defined:

- **app_read** — read-only access (SELECT)
- **app_write** — read + write access (inherits app_read)
- **app_admin** — full administrative privileges (inherits app_write)

These roles represent **permission sets**, not individual users.

3.3.16.4 Database Users

Two actual PostgreSQL users are created:

- **cs2_user**
  - used by the API, ML services, and analytics
  - mapped to the `app_read` role
  - strictly read-only access
- **cs2_admin**
  - used for migrations, schema changes, and data seeding
  - mapped to the `app_admin` role
  - never used by the runtime application

The application never operates under a superuser account, which significantly reduces security risks.

3.3.16.5 Schema-Level Isolation

All application tables are located in a dedicated schema (`cs2`), owned by `cs2_admin`.

This allows: - isolation from the default `public` schema; - fine-grained permission management; - prevention of uncontrolled object creation.

Default privileges are configured so that newly created tables automatically receive correct access rights, eliminating manual permission errors.

3.3.16.6 Application-Level Authorization

On the application side:

- Create / Update / Delete operations are exposed only through an administrative interface;
- Public API endpoints are effectively **read-only**;
- ML and analytics workloads operate exclusively under the `cs2_user` account.

This enforces a clear boundary between: - **data consumers** (API, ML, dashboards), - **data administrators** (schema and reference data management).

3.3.16.7 Security Rationale

This approach provides several advantages:

- eliminates storage of credentials or PII in the database;
- minimizes attack surface;
- follows the principle of least privilege;
- simplifies compliance and auditing.

The absence of user tables is therefore **not a limitation**, but a conscious design choice aligned with the system’s goals and usage patterns.

3.4 Containerization Technical Documentation (Short Version)

3.4.1 Docker Compose

The project is fully containerized using **Docker Compose**, which orchestrates all required services.

3.4.1.1 docker-compose.yml

Defines the following services:

- **cs2-api** – ASP.NET Core production API
- **cs2-api-dev** – ASP.NET Core API with hot reload (dev only)
- **cs2-ml** – FastAPI ML service with CatBoost models
- **cs2-postgres** – PostgreSQL database
- **Volumes** – Persistent DB storage

All services are isolated, configured via environment variables, and connected through a shared Docker network.

3.4.2 Build & Interaction Diagrams

3.4.2.1 Build Pipeline

```
docker compose build
        │
        ├── Build API image
        │   ├── dotnet restore
        │   ├── dotnet publish
        │   └── copy published output
        │
        ├── Pull PostgreSQL image
        │   └── postgres:16-alpine
        │
        └── Build ML image
            ├── install system deps
            ├── pip install requirements
            └── copy FastAPI app + models
```

3.4.2.2 Runtime Interaction Pipeline

```
User / Client
      │
      ▼
ASP.NET Core API (cs2-api)
      │
      ├── PostgreSQL (cs2-postgres)
      │     └── reference & metadata queries
      │
      └── ML Service (cs2-ml)
            └── price prediction
```

3.4.2.3 Request Flow

1. Client sends request to API.
2. API reads metadata from PostgreSQL.
3. API sends feature JSON to ML service.
4. ML service returns predicted price.
5. API responds to client.

3.4.3 Docker Images Overview

3.4.3.1 API Image — cs2price_prediction-api

- ASP.NET Core 8 Web API.
- Handles requests, DB access, ML communication.
- Multi-stage build:
  - Build: `dotnet/sdk:8.0`
  - Runtime: `dotnet/aspnet:8.0`
- Final size: **~335 MB**
- Optimized with multi-stage build, `.dockerignore`, env secrets.
- Exposes port **8080**.

3.4.3.2 ML Image — cs2price_prediction-ml-service

- FastAPI service with **6 CatBoost models**.
- Image: `python:3.11-slim`
- Final size: **~1.3 GB** (ML dependencies).

- Optimized pip/apt usage, non-root user.
- Exposes port **8000**.

3.4.3.3 Database Image — postgres:16-alpine

- Stores skins, patterns, stickers, and metadata.
- Uses persistent volume `db-data`.
- Final size: **~395 MB**.
- Credentials via `.env`.

3.4.4 Development Container

3.4.4.1 cs2-api-dev

- Development-only.
- `dotnet watch` hot reload.
- Image: `dotnet/sdk:8.0`
- Size: **~1.19 GB**.

3.4.5 Containers & Ports

Container	Role	Port
cs2-api	Main API	8087 → 8080
cs2-ml	ML inference	8000
cs2-postgres	Database	internal

API healthcheck: `/health` endpoint.

3.4.6 Performance Summary

- Total RAM usage: **600–800 MB**
- Cold start: **12–15 s**
- Prediction latency: **< 50 ms**

3.4.7 Conclusion

The Docker-based architecture is modular, efficient, and suitable for both production and development, fully meeting diploma technical requirements.

3.5 ML models Technical Documentation

3.5.1 Data Collection

3.5.1.1 Context and Research Motivation

Data collection was the **most complex, time-consuming, and critical stage** of this project. Unlike many traditional machine learning problems, the CS2 skin market does not provide open, standardized, or transparent access to historical transaction data.

There is **no official Valve API** that exposes completed sales or realized transaction prices. Most publicly available sources provide only:

- active marketplace listings;
- seller-defined asking prices;
- approximate or aggregated price indicators.

Such data reflects **seller intent rather than actual market outcomes**. Many listed items are never sold at the displayed price, remain unsold for long periods, or are repeatedly relisted with changing values.

Using such sources would introduce **systematic bias**, high noise, and distorted learning signals, making them unsuitable for supervised machine learning models that aim to predict real market prices.

As a result, reliable data acquisition became a fundamental research challenge rather than a simple preprocessing step.

3.5.1.2 Limitations of Public CS2 Market Data

Publicly accessible CS2 price sources suffer from several critical limitations:

1. **Lack of transaction finality**
   Listing prices do not represent completed buyer–seller agreements.

2. **Survivorship bias**
   Unsold or overpriced items remain visible, while completed sales disappear.

3. **Price stickiness**
   Listing prices may remain unchanged despite shifts in demand.

4. **Absence of attribute-level pricing**
   Public data rarely links prices to float values, patterns, or sticker composition.

These limitations make public data sources fundamentally incompatible with accurate price modeling.

3.5.1.3 Partnership-Based Data Acquisition (DMarket)

To overcome these constraints, this project relied on **direct cooperation with the project partner — DMarket**, one of the largest CS2 marketplace platforms.

This partnership enabled access to **real completed transaction data**, which is rarely available for academic research in virtual item markets.

The dataset consists **exclusively of finalized sales**, meaning:

- each record corresponds to a confirmed transaction;
- prices reflect actual buyer–seller agreement;
- no speculative or placeholder values are present.

This approach ensures that the target variable (`price`) represents **true market-clearing prices**.

3.5.1.4 Strict Data Inclusion and Exclusion Policy

To preserve dataset integrity, a strict filtering policy was applied.

3.5.1.4.1 Included Data

Data Type	Included
Completed transactions	Yes
Real sold prices	Yes

3.5.1.4.2 Explicitly Excluded Data

Data Type	Excluded
Active listings	No
Asking prices	No
Unsold offers	No
Synthetic or generated data	No
Manual price estimates	No

This policy eliminates speculative bias and significantly improves the reliability of the learning signal.

3.5.1.5 Data Scope and Attribute Granularity

Each transaction record contains detailed attribute-level information, including:

- weapon type and specific weapon name;
- skin name and cosmetic category;
- wear tier (categorical);
- exact float value (continuous);
- StatTrak flag;
- pattern identifiers (where applicable);
- sticker configuration and composition;
- final transaction price in USD.

Such **fine-grained attribute representation** is essential, as CS2 skin pricing is driven by complex interactions between cosmetic attributes rather than simple additive effects.

3.5.1.6 Temporal Coverage and Market Dynamics

The collected data spans multiple market periods and naturally reflects:

- short-term price fluctuations;
- effects of in-game updates and balance changes;
- release of new cases and skins;
- shifts in player demand and trading behavior.

As a result, the dataset exhibits **non-stationarity**, which is a realistic property of real-world financial and virtual item markets.

This characteristic was explicitly considered during model selection and evaluation.

3.5.1.7 Data Quality vs. Data Volume Trade-off

Due to restricted access to completed transactions, the final dataset is **moderate in size but high in quality**.

This trade-off was intentional:

- fewer samples;
- lower label noise;
- higher signal-to-noise ratio;
- improved model generalization.

For market price prediction tasks, **data quality dominates raw volume**.

3.5.1.8 Data Validation and Consistency Checks

Before further analysis, the dataset underwent multiple validation steps:

- verification of numeric ranges (float values $\in [0, 1]$);
- consistency checks between wear tiers and float values;
- validation of pattern identifiers;
- removal of duplicate or malformed records.

These checks ensured internal consistency and prevented data leakage.

3.5.1.9 Ethical and Legal Considerations

All data collection and usage were conducted responsibly:

- no personal user data was collected;
- no private identifiers were accessed;
- no individual trading behavior can be reconstructed;
- data usage complies with marketplace policies;
- the dataset is used strictly for academic research.

The dataset reflects **aggregated market behavior** without enabling identification of individual participants.

3.5.1.10 Limitations of the Dataset

Despite its high quality, the dataset has unavoidable limitations:

- limited samples for extremely rare items;
- uneven distribution across price ranges;
- market volatility and non-stationarity;
- dependence on partner-provided access.

These limitations were explicitly considered during modeling and evaluation.

3.5.1.11 Data Collection Summary

The data collection phase required substantially more effort than model training itself. However, it resulted in a **realistic, unbiased, and production-grade dataset** based exclusively on **real completed transactions from DMarket**.

This dataset provides a solid foundation for exploratory data analysis, feature engineering, and machine learning modeling, enabling predictions based on **actual CS2 market behavior rather than speculative approximations**.

3.5.2 EDA Methodology

3.5.2.1 Purpose of Generalized EDA

Exploratory Data Analysis (EDA) in this project was not performed as a single, one-size-

fits-all procedure. Instead, EDA was conducted **separately for each model-specific dataset** corresponding to different CS2 item categories.

The primary goal of EDA was to **identify meaningful features, understand price formation mechanisms, and justify modeling decisions** prior to training machine learning models.

A total of **six independent EDA pipelines** were executed for the following datasets:

- Doppler knives
- Fade knives
- Fade weapons
- Case Hardened knives
- Case Hardened weapons
- Float-sensitive weapons

While the concrete features and visualizations differ between categories, the analytical methodology remained consistent across all datasets.

3.5.2.2 Unified EDA Workflow

For each dataset, EDA followed a structured and repeatable workflow designed to ensure comparability of conclusions and reproducibility of results.

The following analytical stages were applied to every dataset.

3.5.2.3 Dataset Structure and Feature Composition

At the initial stage, each dataset was inspected to determine:

- number of observations;
- feature composition;
- data types (numerical vs categorical);
- presence of category-specific attributes (e.g., phase, pattern, fade rank).

This step ensured that each dataset was suitable for supervised regression and helped identify features that were unique to specific item categories.

3.5.2.4 Missing Values and Data Completeness

Missing values were analyzed for all features.

Importantly, missing values in CS2 market data usually represent **absence of a property** rather than data corruption (e.g., absence of stickers or non-applicable patterns).

Based on this observation:

- numerical features were filled with `0.0`;
- categorical features were filled with `"unknown"`.

No dataset exhibited systematic missingness that could introduce bias or require complex imputation strategies.

3.5.2.5 Target Variable Analysis

For each dataset, the distribution of the target variable (`price`) was analyzed using histograms and boxplots.

Across all categories, prices exhibited:

- strong right-skewness;
- heavy-tailed distributions;
- presence of rare, high-priced outliers.

Outliers were retained, as they represent economically meaningful events and are critical for realistic market modeling.

This analysis motivated the use of **robust error metrics** and discouraged assumptions of normality.

3.5.2.6 Categorical Feature Analysis

Categorical features such as weapon type, cosmetic phase, pattern group, and wear tier were analyzed individually for each dataset.

EDA consistently revealed:

- strong imbalance between common and rare categories;
- significant differences in average price across categories;
- category-dependent variance in price.

These findings indicated that categorical variables play a dominant role in price formation and must be handled natively by the learning algorithm.

3.5.2.7 Numerical Feature Analysis

Numerical features, most notably float value and sticker-derived attributes, were analyzed for distributional properties and relationship with price.

Across all datasets:

- float values exhibited non-linear relationships with price;
- sticker features showed high variance and weak linear correlation;
- identical numerical values could correspond to a wide range of prices.

This confirmed that numerical features interact strongly with categorical context and cannot be modeled independently.

3.5.2.8 Feature Interaction Analysis

EDA consistently identified strong interaction effects, including:

- weapon type × float value;
- cosmetic phase or pattern × weapon category;
- wear tier × float value;
- float value × sticker configuration.

These interaction patterns demonstrate that CS2 price formation is inherently non-linear and context-dependent.

3.5.2.9 Implications for Feature Engineering

Insights obtained during EDA directly informed feature engineering decisions:

- float values were preserved as continuous variables;
- categorical features were not aggressively encoded or reduced;
- interaction effects were intentionally preserved;
- outliers were retained to reflect real market behavior.

3.5.2.10 Implications for Model Selection

Based on EDA results, the following modeling decisions were justified:

- rejection of purely linear models as primary predictors;
- preference for tree-based gradient boosting algorithms;
- segmentation of the problem into category-specific models;
- prioritization of robustness over strict distributional assumptions.

These conclusions directly motivated the choice of **CatBoost** as the primary modeling algorithm.

3.5.2.11 Reproducibility and Detailed EDA Reports

A complete, detailed EDA (including visualizations and dataset-specific conclusions) was performed separately for each of the six datasets.

Due to space constraints, only the generalized EDA methodology is presented here.

Full EDA reports for each dataset-model pair are available in the project repository, where all figures, intermediate analyses, and category-specific findings can be reviewed in detail.

3.5.2.12 EDA Summary

Exploratory Data Analysis was conducted as a systematic, repeatable process across all six datasets used in this project.

Although the concrete characteristics of each dataset differ, the unified EDA methodology ensured consistent feature discovery, robust modeling assumptions, and reproducible analytical conclusions.

3.5.3 Modeling Methodology

3.5.3.1 Modeling Objectives

The primary objective of the modeling stage was to develop machine learning models capable of accurately predicting CS2 skin prices based on real market transaction data.

Given the heterogeneity of CS2 items and the complexity of price formation mechanisms, the modeling process focused on:

- capturing non-linear relationships;
- handling high-cardinality categorical features;
- maintaining robustness to outliers;
- ensuring interpretability and reproducibility;
- achieving low inference latency suitable for production use.

3.5.3.2 Rationale for Multiple Specialized Models

A key architectural decision of this project was to **avoid a single global model** and instead train **multiple specialized models**, each optimized for a specific item category.

This decision was motivated by the following observations:

- different item categories rely on different dominant price drivers;
- feature importance varies significantly across categories;
- price distributions differ substantially between weapons and skin types;
- a global model risks underfitting rare but valuable segments.

As a result, six independent models were trained:

- Doppler knives
- Fade knives
- Fade weapons
- Case Hardened knives
- Case Hardened weapons
- Float-sensitive weapons

Each model was trained, optimized, and evaluated independently.

3.5.4 Baseline Models

3.5.4.1 General Training Strategy

The modeling stage aimed to construct accurate, robust, and interpretable machine learning models for CS2 skin price prediction using real completed transaction data.

Given the strong heterogeneity of the CS2 market, a **category-specific modeling strategy** was adopted. Instead of training a single global model, independent models were trained for each market segment.

This approach allows: - capturing category-specific price drivers; - reducing bias toward dominant item groups; - improving accuracy for rare but valuable items.

All models were trained using supervised regression on historical transaction data.

3.5.4.2 Baseline Models

Two baseline approaches were used consistently across all categories.

3.5.4.2.1 Linear Regression Baseline

A linear regression model with one-hot encoded categorical features was trained as a classical baseline.

Purpose: - provide a reference point; - test whether linear assumptions are sufficient.

Across all categories, linear regression failed to capture non-linear price behavior and systematically underestimated expensive items.

3.5.4.2.2 CatBoost Baseline

CatBoost was selected as the primary non-linear baseline model due to: - native categorical feature handling; - robustness to outliers; - strong performance on tabular data; - minimal preprocessing requirements.

Baseline CatBoost models used conservative hyperparameters to avoid overfitting.

3.5.4.3 Hyperparameter Optimization (Optuna)

Hyperparameter optimization was performed using **Optuna**, which applies Bayesian optimization to efficiently search the parameter space.

Optimized parameters included: - tree depth; - learning rate; - number of boosting iterations; - L2 leaf regularization.

The optimization objective was **RMSE**, as large errors on high-priced items have higher economic cost.

Each optimization was limited to approximately **20 trials** to balance performance gains and computational cost.

3.5.4.4 Category-Specific Training Results

3.5.4.4.1 float_sensitive_weapons

Model	MAE	RMSE	R^2	Inference Time (s)
CatBoost Baseline	**192.94**	**739.91**	**0.6783**	0.00334
CatBoost Tuned	197.94	789.79	0.6335	0.00334
Linear Regression	301.63	1178.02	0.1845	0.00489

Analysis:
Baseline CatBoost achieved the best overall performance. Hyperparameter tuning did not improve results, indicating that float is already well-captured by the baseline configuration.

Selected model: CatBoost Baseline

3.5.4.4.2 fade_weapon

Model	MAE	RMSE	R^2	Inference Time (s)
CatBoost Baseline	624.86	2938.32	0.5374	0.00265
CatBoost Tuned	**606.06**	**2834.46**	**0.5696**	0.00160
Linear Regression	987.93	4760.43	-0.2141	0.00457

Analysis:
Hyperparameter tuning significantly improved performance. Despite optimization, fade weapons remain difficult to model due to strong visual variability.

Selected model: CatBoost Tuned

3.5.4.4.3 fade_knives

Model	MAE	RMSE	R^2
CatBoost Baseline	532.65	3129.10	0.7256
CatBoost Tuned	**468.74**	**2876.82**	**0.7680**
Linear Regression	1239.98	5551.91	0.1361

Analysis:
Optuna optimization reduced RMSE by approximately 250 USD and increased explained variance. Linear regression completely fails to model fade knife pricing.

Selected model: CatBoost Tuned

3.5.4.4.4 doppler_knives

Model	MAE	RMSE	R^2
CatBoost Baseline	310	573	0.887
CatBoost Tuned	**304**	**563**	**0.891**
Linear Regression	642	991	0.663

Analysis:
Baseline CatBoost already performed strongly. Hyperparameter tuning provided small but consistent improvements.

Selected model: CatBoost Tuned

3.5.4.4.5 ch_knives

Model	MAE	RMSE	R^2
CatBoost Baseline	335.90	816.42	0.9841
CatBoost Tuned	**178.98**	**564.46**	**0.9924**
Linear Regression	2189.06	6062.03	0.1214

Analysis:
Hyperparameter optimization dramatically improved performance. Linear regression severely underfits the data.

Selected model: CatBoost Tuned

3.5.4.4.6 case_hardened_gun

Model	MAE	RMSE	R^2
CatBoost Tuned	**446.36**	**2645.68**	**0.6243**
Linear Regression	1012.46	4272.35	0.1017

Analysis:
Optimization reduced RMSE by over 900 USD compared to baseline. Case Hardened guns remain inherently volatile due to rare pattern effects.

Selected model: CatBoost Tuned

3.5.4.5 Model Selection Summary

Category	Selected Model	Reason
float_sensitive_weapons	CatBoost Baseline	Best RMSE; tuning unnecessary
fade_weapon	CatBoost Tuned	Improved RMSE and R^2
fade_knives	CatBoost Tuned	Significant optimization gains
doppler_knives	CatBoost Tuned	Highest overall accuracy
ch_knives	CatBoost Tuned	Near-perfect fit
case_hardened_gun	CatBoost Tuned	Large RMSE reduction

3.5.4.6 Reproducibility and Experiment Tracking

All experiments, hyperparameter searches, and evaluation metrics were tracked using **Weights & Biases (W&B)**.

Public experiment dashboards: - https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-fade-knives-pricing
- https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-ch-knife-pricing
- https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-ch-pricing
- https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-fade-pricing
- https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-fade-weapons-pricing
- https://wandb.ai/20220481-https-en-ehuniversity-lt-/cs2-doppler-knives-pricing

Complete training pipelines, feature engineering code, and model artifacts are available in the project repository.

3.6 API Technical Documentation

3.6.1 API Architecture Overview

The **cs2price_prediction** system consists of several logically separated APIs. Each API is responsible for a specific functional area and follows a clear separation between user-facing and internal system components.

3.6.1.1 API Groups

API	Purpose	Accessibility
Meta API	Reference and metadata access	Public
Prediction API	Price prediction workflow	Public
AI Explanation API	Prediction interpretability	Public
Admin API	Administrative management	Internal
ML API	Machine learning inference	Internal

This document provides detailed documentation for all public APIs. Internal APIs are described briefly with references to the project repository.

3.6.2 Meta API Documentation

3.6.2.1 Overview

The Meta API provides read-only reference data related to CS2 weapons, skins, cosmetic attributes, and stickers. It is designed to be used by pricing services, analytics modules, and machine learning pipelines.

All responses are returned in **JSON** format.

Base URL: `/api/v1/meta`
Authentication: Not required

3.6.2.2 Architecture Context

The Meta API acts as a centralized reference data provider and does not store user-specific or transactional data. It supplies normalized metadata used across the system.

3.6.2.3 Error Handling

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable error description"
  }
}
```

HTTP Code	Meaning
400	Invalid request
404	Resource not found
500	Internal server error

3.6.2.4 Endpoints

- **GET /weapon-types** – returns all weapon types
- **GET /weapon-types/{weaponTypeId}/weapons** – returns weapons by type
- **GET /weapons/{weaponId}/skins** – returns skins for a weapon
- **GET /skins/{skinId}/wear-tiers** – returns wear tiers
- **GET /skins/{skinId}/patterns** – returns pattern IDs
- **GET /stickers?q=&limit=** – searches stickers

3.6.3 Prediction API Documentation

3.6.3.1 Overview

The Prediction API is the main entry point for generating a unified CS2 skin price prediction. It aggregates metadata, validates input, selects the appropriate ML model, invokes the ML API, and returns a structured prediction result.

All responses are returned in **JSON** format.

Base URL: `/api/v1`
Authentication: Not required

3.6.3.2 POST /predict

Performs a complete price prediction workflow.

3.6.3.2.1 Responsibilities

- Input validation
- Metadata resolution
- ML model selection
- ML API invocation
- Response aggregation

3.6.4 AI Explanation API Documentation

3.6.4.1 Overview

The AI Explanation API provides natural-language explanations describing how different attributes influence the predicted price.

It improves transparency and interpretability of machine learning predictions.

Base URL: `/api/v1/ai`
Authentication: Not required

3.6.4.2 Endpoints

- **POST /explain** – concise explanation
- **POST /explain-v2** – extended explanation

3.6.5 Admin API (Summary)

3.6.5.1 Purpose

The Admin API provides administrative and maintenance functionality such as configuration management and system monitoring.

This API is internal and requires authentication.

Full documentation is available in the project repository.

3.6.6 ML API (Summary)

3.6.6.1 Purpose

The ML API is an internal machine learning microservice responsible for executing model inference for trained models.

It is consumed exclusively by the Prediction API.

Full documentation is available in the project repository.

3.7 Qualitative and Quantitative Testing Technical Documentation

3.7.1 Introduction

Within the scope of the academic discipline **“Qualitative or Quantitative Testing”**, the **cs2price_prediction** system was evaluated using a combined testing approach. The objective of this testing phase was to assess the system from both quantitative and qualitative perspectives, covering machine learning accuracy, API correctness, performance, and usability.

The system under test is a machine learning–based solution designed to predict market prices of CS2 skins and expose the prediction logic via a REST API. Due to the complexity and volatility of the CS2 market, comprehensive testing was required to validate both numerical performance and user-facing behavior.

3.7.2 Quantitative Testing

3.7.2.1 Quantitative Testing of Machine Learning Models

Quantitative testing was primarily applied to evaluate the accuracy and stability of the machine learning models. Instead of a single generalized model, multiple **specialized regression models** were trained to account for different pricing mechanics across item categories.

The following standard regression metrics were used:

- **MAE (Mean Absolute Error)** — average absolute prediction error
- **RMSE (Root Mean Squared Error)** — penalizes large deviations and reflects model stability
- **R^2 (Coefficient of Determination)** — measures explanatory power of the model

These metrics provide an objective numerical assessment of model performance.

3.7.2.2 Overview of Model Performance

Six specialized models were evaluated, each trained on a specific subset of weapon or skin types:

Model	MAE	RMSE	R^2	Interpretation
doppler_knives	304.15	563.21	**0.8909**	High accuracy; low error relative to price variance; very good fit.
fade_weapon	606.06	2834.46	**0.5696**	Moderate accuracy; high variance in fade weapons reduces model stability.
ch_knives	335.90	816.42	**0.9841**	Excellent performance; very strong model fit.
case_hardened_gun	446.35	2645.68	**0.6243**	Moderate performance; high variability of CH patterns increases error.
fade_knives	468.73	2876.82	**0.7680**	Good performance; complexity caused by pattern dependency.
float_sensitive_weapons	192.94	739.91	**0.6783**	Lowest MAE; effective for float-driven pricing.

The results demonstrate **good to excellent predictive quality** across most categories. Knife-based models show the strongest performance, while fade-based weapons exhibit higher error due to intrinsic market volatility.

3.7.2.3 Quantitative Testing of API Performance

Quantitative testing was also applied to evaluate non-functional requirements, particularly performance:

- Metadata endpoints: **3–4 ms average response time**
- Prediction endpoint: **5–6 ms average response time**
- Explanation endpoints: **~2000 ms average response time**

The increased latency of explanation endpoints is expected due to additional reasoning and feature attribution logic.

3.7.3 Qualitative Testing

Quantitative metrics alone are insufficient to fully evaluate a user-facing system. Therefore, qualitative testing was conducted to assess correctness, usability, and consistency.

3.7.3.1 Qualitative Evaluation Criteria

Qualitative testing focused on the following aspects:

- Correctness and completeness of API responses
- Stability under invalid or malformed input
- Clarity and usefulness of error messages
- Semantic relevance of explanation texts
- Consistency of response formats across endpoints

This evaluation was performed from the perspective of an end user interacting with the system.

3.7.4 Test Cases and Postman-Based Testing

3.7.4.1 User-Accessible Endpoints

Test cases were created for **all endpoints with which a user can interact**, including:

- `/api/predict`
- `/api/ai/explain`
- `/api/ai/explain-v2`
- `/api/meta/*` (weapons, skins, wear tiers, patterns, stickers)

3.7.4.2 Test Case Design

Each test case includes:

- Input data and preconditions
- Expected output or system behavior
- Error scenarios and boundary conditions

The test cases cover:

- Positive scenarios (valid input)
- Negative scenarios (missing or invalid fields)
- Boundary values (extreme float values, empty sticker arrays)
- Schema and type validation
- Error handling consistency

Each test case is mapped to functional requirements, ensuring traceability.

3.7.4.3 Test Case Implementation Using Postman

All test cases were **implemented as automated Postman collections**. Postman was selected due to its support for REST API testing and automation.

Postman test scripts were used to:

- Validate HTTP status codes
- Verify response schemas
- Assert business rules (e.g., predicted price ≥ 0)
- Measure response times

Separate collections were created for Predict, Explain, and Meta endpoints.

3.7.4.4 Test Execution Results

- Total automated test cases: **84**
- Total endpoints covered: **100%**
- Passed tests: **84**
- Failed tests: **0**
- Overall pass rate: **100%**

These results confirm correct and stable behavior of all user-facing endpoints.

3.7.5 Integrated Testing Results

The combined application of qualitative and quantitative testing provides a comprehensive system evaluation:

- Quantitative testing confirms **accuracy, stability, and performance**
- Qualitative testing confirms **correctness, usability, and consistency**
- Automated testing ensures **reproducibility and reliability**

This integrated approach fully соответствует objectives of the discipline **“Qualitative or Quantitative Testing”**.

3.7.6 Conclusion Qualitative and Quantitative Testing

The conducted qualitative and quantitative testing demonstrates that the **cs2price_prediction** system meets all defined functional and non-functional requirements. The machine learning models achieve strong predictive performance, while the REST API provides reliable and user-consistent interaction.

All detailed test cases, Postman collections, quantitative metrics, and testing reports are available in the project repository, where the testing process is documented in greater detail.

4 User Guide Documentation

4.1 User Guide

This section describes how end users interact with the CS2 Skin Price Prediction system. It explains the main user flows, available features, and expected behavior of the application.

The system is designed to be **simple for users**, while hiding significant technical and machine learning complexity behind a clean interface.

4.1.1 Contents

- Features Walkthrough
- FAQ & Troubleshooting

4.1.2 Getting Started

4.1.2.1 System Requirements

Requirement	Minimum	Recommended
Browser	Chrome 90+, Firefox 88+, Safari 14+	Latest version
Screen Resolution	1280×720	1920×1080
Internet Connection	Required	Stable broadband
Device	Desktop	Desktop

The application is optimized for desktop usage, as skin analysis and comparison benefit from larger screen sizes.

4.1.2.2 Accessing the Application

1. Open a supported web browser
2. Navigate to the application URL
3. The main interface loads immediately — no installation required

The system is fully web-based and does not require any client-side setup.

4.1.2.3 First Launch Overview

After opening the application, the user is presented with the main prediction interface.

The screen is logically divided into: - **Item selection panel** (weapon, skin, wear) - **Attribute input fields** (float, pattern, stickers, special flags) - **Prediction output section** (estimated price and confidence context)

The interface is intentionally minimalistic to reduce cognitive load and allow users to focus on price evaluation.

4.1.3 Typical User Flow

1. Select weapon and skin
2. Specify wear and float (if applicable)
3. Add optional attributes (StatTrak™, stickers, pattern details)
4. Submit request
5. Receive predicted market price

All predictions are generated in real time using the deployed ML models.

4.1.4 User Roles

Role	Description
Guest User	Can freely use all prediction features
Advanced User	Uses the system for trading, analysis, or research

The system does not require authentication, as no personal data is stored.

4.2 Feature Walkthrough Documentation

This document explains the core features available to end users and how they interact with the CS2 Skin Price Prediction system.

4.2.1 Feature: Skin Price Prediction

4.2.1.1 Overview

The primary feature of the application is **predicting the fair market price** of a CS2 skin based on historical real sales data.

Unlike listing-based tools, this system estimates **what the item is likely to sell for**, not what sellers are asking.

4.2.1.2 How to Use

Step 1: Select weapon type
Choose the base weapon (e.g. AK-47, AWP, Knife).

Step 2: Select skin and wear
Specify the skin name and wear tier (FN, MW, FT, etc.).

Step 3: Enter float and additional attributes
Optionally enter float value, StatTrak flag, sticker information, or pattern-related data.

Step 4: Submit for prediction
Click the prediction button to receive an estimated market price.

4.2.1.3 Expected Result

The system returns: - predicted price in USD - category-specific evaluation logic - stable result independent of current listings

4.2.1.4 Tips

- More detailed inputs lead to more accurate predictions
- Float-sensitive skins benefit significantly from precise float values
- Stickers and rare patterns noticeably affect the output

4.2.2 Feature: Category-Aware Modeling

4.2.2.1 Overview

The system automatically routes each request to a **specialized ML model** depending on the selected skin category.

This avoids the inaccuracies of a single universal model.

4.2.2.2 How It Works

- Case Hardened items use pattern-aware models
- Fade items use gradient-based features
- Float-sensitive items emphasize wear precision

This logic is transparent to the user but critical for accuracy.

4.2.3 Feature: Real-Time Inference

4.2.3.1 Overview

All predictions are performed in real time.

4.2.3.2 Characteristics

- Average response time: under 50 ms
- Models fully loaded in memory
- No queueing or delayed responses

This allows the system to be used interactively during trading or analysis.

4.2.4 Feature: No Account Required

4.2.4.1 Overview

The application does not require registration or login.

This design choice: - lowers entry barrier - avoids personal data storage - simplifies compliance and security

4.2.5 Feature Comparison

Feature	Available
Price Prediction	Yes
Real sales-based data	Yes
Pattern-aware pricing	Yes
Float-sensitive modeling	Yes
Account required	No

4.3 FAQ & Troubleshooting Documentation

4.3.1 Frequently Asked Questions

4.3.1.1 General

Q: Where do the prices come from?

A: All predictions are based on real completed market sales. Listings and asking prices are intentionally not used.

Q: Why does the predicted price differ from current listings?

A: Listings often reflect seller expectations, not actual market value. The system predicts realistic sale prices.

Q: Is this an official Valve or Steam service?

A: No. This is an independent academic and technical project based on publicly observable market behavior.

4.3.1.2 Accuracy & Models

Q: Why does the same skin sometimes give different prices?

A: Small changes in float, wear, or attributes can significantly affect value, especially for rare or float-sensitive items.

Q: Why are there multiple ML models?

A: Different skin categories have fundamentally different pricing logic. Specialized models improve accuracy and stability.

4.3.1.3 Troubleshooting

Problem	Possible Cause	Solution
Prediction seems too low	Missing attributes	Add float, stickers, or pattern info
Prediction seems too high	Rare pattern detected	Verify entered pattern or attributes
Page not responding	Network issue	Refresh page and retry
Unexpected value	Incorrect wear/float	Double-check inputs

4.3.1.4 Error Messages

Message	Meaning	Resolution
`Invalid input`	Missing or malformed data	Review input fields
`Prediction unavailable`	Model routing issue	Retry request
`Timeout`	Network latency	Refresh and try again

5 Retrospective

5.1 Retrospective

This section reflects on the development of the **CS2 Skin Price Prediction System**,

summarizing achieved results, encountered limitations, architectural trade-offs, and lessons learned. The project was implemented as a **production-ready ML system**, covering the full lifecycle from raw data processing to stable inference.

5.1.1 Project Status Overview

By the end of the research phase, the project has reached a **stable and fully functional state** with the following:

- A **working production system** capable of predicting prices for selected CS2 skin families.
- **Six independent machine learning models**, each tailored to a specific skin type or family.
- Models demonstrate **high predictive performance** on validation data and align well with real market behavior.
- The system operates on a **static dataset and fixed models**, without daily retraining or live data updates.
- This limitation is caused by **data provider NDA restrictions**, which prohibit automated continuous data ingestion.
- The current version of the system is considered **stable and fixed until September 25, 2025**, defining a clear research boundary.

5.1.2 What Went Well

5.1.2.1 Technical Successes

- Adoption of **multiple specialized ML models** significantly outperformed a single universal predictor.
- **CatBoost** proved highly effective for tabular data with high-cardinality categorical features.
- Full ML lifecycle implemented: data acquisition → EDA → feature engineering → training → evaluation → deployment.
- **Low-latency inference (<50 ms)** achieved, suitable for real-time usage.
- Clean separation via a **FastAPI-based ML microservice**.
- Model behavior validated using **SHAP** for interpretability.

5.1.2.2 Process Successes

- EDA strongly influenced architectural decisions.
- Iterative experimentation enabled fast validation of assumptions.
- Continuous documentation improved system maintainability.
- Focus on production realism over purely academic metrics.

5.1.2.3 Personal Achievements

- Practical experience with **real-world ML system design**.
- Improved handling of noisy and incomplete market data.
- Stronger ability to justify ML decisions technically and economically.
- Enhanced communication of ML concepts to non-technical audiences.

5.1.3 What Didn't Go as Planned

Planned	Actual Outcome	Cause	Impact
Unified ML model	6 specialized models	Domain heterogeneity	Medium
Continuous retraining	Static models	NDA restrictions	Medium
Large datasets	Smaller, cleaner data	Limited access to sales	Low
Simple ingestion	Complex pipeline	No public API	High

5.1.3.1 Key Challenges

1. **Data Availability**
   - No official API for historical sold prices.
   - Reliance on third-party marketplace data.
   - Strict filtering to preserve label quality.
2. **Market Volatility**
   - Prices affected by updates, hype, and case changes.

- Required robust models over short-term trend fitting.

3. **NDA-Driven Constraints**
   - Automated updates and retraining not permitted.
   - Resulted in a deliberately static but stable system.

5.1.4 Technical Debt & Known Limitations

ID	Issue	Severity	Description	Potential Improvement
TD-001	Static datasets	Medium	No automatic updates	Scheduled retraining
TD-002	Manual retraining	Medium	No ML CI/CD	Automated pipelines
TD-003	Feature duplication	Low	Repeated schemas	Feature registry

5.1.5 Future Development Directions

5.1.5.1 Short-Term

1. Extend data preparation to **additional skin families**.
2. Introduce **prediction confidence intervals**.

5.1.5.2 Long-Term

3. Integrate pricing service into **skin trading platforms**.
4. Develop a **standalone pricing service** for end users.

5.1.6 Lessons Learned

5.1.6.1 Technical

- Data quality is more important than data volume.
- Domain specialization significantly improves accuracy.
- Interpretability is essential for trust.
- ML systems must be designed with production constraints in mind.

5.1.6.2 Process

- EDA should precede architecture decisions.
- Early assumptions must be validated quickly.
- Documentation is a core part of system stability.

6 Final Conclusion

This thesis explored the use of machine learning methods for estimating the market value of virtual items, using Counter-Strike 2 (CS2) skins as a case study. The relevance of this work is driven by the high volatility of the in-game item market and the lack of transparent pricing tools.

Achieved Results

During the course of the project, the following results were achieved:

1. **Market Data Collection and Preparation**
   A large dataset of CS2 skin market data was collected and processed in cooperation with a partner trading platform. Due to non-disclosure agreements (NDA), the dataset does not include exact timestamps of individual sales; however, the available data proved sufficient for reliable analysis and model training.

2. **Machine Learning Models**
   Multiple machine learning models were trained on a substantial volume of high-quality market data. The use of several specialized models instead of a single universal approach resulted in improved robustness and stable predictive performance for selected skin categories.

3. **Application Logic and Data Storage**
   The full logic for user interaction with the prediction system was implemented, including request handling, access to trained models, and result delivery. A structured relational database was designed to support data storage and system operations.

4. **System Architecture**
   A functional application architecture was developed, providing a foundation for integrating data processing, model inference, and user-facing components within a unified system.

Limitations and Non-Implemented Components

Despite the achieved results, several components were intentionally not implemented within the scope of the diploma project:

1. **Continuous Data Updates**
   The system does not perform continuous or live data updates. This limitation is directly related to restrictions imposed by the data provider and the terms of the NDA, which prevent the disclosure and use of continuously refreshed market data in an academic context.

2. **Regular Model Retraining**
   Automated periodic retraining of models (e.g., on a 24-hour cycle) was not enabled, as it requires continuous access to updated transactional data, which was not permitted under the existing data-sharing agreements.

Temporal Scope and Validity

As a result of these constraints, the developed solution represents a stable and reproducible research snapshot of the CS2 skin market, valid **until September 25, 2025**. This clearly defined temporal scope ensures transparency regarding the applicability and limitations of the presented results.

Overall Assessment

- The project meets its research objectives and demonstrates the practical applicability of machine learning for price estimation in digital markets.
- The dataset collection strategy proved to be appropriate and sufficient for training predictive models.
- Accurate price prediction is feasible for selected CS2 skin families.
- Model performance is limited by market volatility, data availability, and concept drift.
- Significant market changes require additional time and data for effective model adaptation.